

IBenchmark process

TTR Data Science

29 September, 2026

Table of Contents

1	1 Table of contents	3
2	1. What purpose does the component serve? What problem does it solve?	4
3	2. Where does the component belong? What part of the process is it located?	5
4	3. Component/Method description	6
4.1	3.1 Manual suffix instructions and governance.....	6
4.2	3.2 Reference price calculation (mean/median update).....	7
4.2.1	Inputs	7
4.3	3.3 Mean price stability (Dynamic vs. Stable Price)	8
4.4	3.4 Multipack detection	9
4.4.1	Naïve Bayes theorem and posterior probability.....	9
4.5	3.5 Multi-piece analysis.....	11
4.6	3.6 Change maker	12
4.7	3.7 Bundle-pack analysis (BLP)	13
4.8	3.8 Price-suffix cross-coding instructions	15
4.9	3.9 Median price enhancements and bins approach	18
5	4. Inputs	21
6	5. Outputs	22
7	6. Best practices and checklist	23

1 1 Table of contents

1. **What purpose does the component serve? What problem does it solve?**
2. **Where does the component belong? What part of the process is it located?**
3. **Component/Method description**
 - Manual suffix instructions and governance
 - Reference price calculation (mean/median update)
 - Mean price stability (Dynamic vs. Stable Price)
 - Multipack detection
 - Multi-piece analysis
 - Change maker
 - Bundle-pack analysis (BLP)
 - Price-suffix cross-coding instructions
 - Median price enhancements and bins approach
4. **Inputs**
5. **Outputs**
6. **Best practices and checklist**

2 1. What purpose does the component serve? What problem does it solve?

The **IBenchmark (IB)** process is the first item-level validation stage in the SIRVAL scanning flow. Its primary role is to detect and correct mismatches between the **sales value** and **units** reported by retailers. Such mismatches commonly occur when suppliers report the value of a **multipack** (e.g., a six-pack of soda) but count the units as if they were individual items; a six-pack priced at \$6 could be logged as six units at \$1 each. This mis-reporting inflates unit volumes and deflates prices. IB resolves the inconsistency by comparing each item's **raw price** to a **reference price** and, when a mismatch is detected, applies a **conversion factor** (price suffix) so that volumes and prices are corrected. The process runs for each item in each store/period, making IB the first line of defense against pack size errors in scanning data.

Example (six-pack misreporting) – Consider a retailer that sells a 6-pack of soda for \$6.00. When reported correctly the value (\$6.00) and unit count (1 pack) align. If the retailer instead logs the units as **6** (counting each bottle), the raw unit price appears to be \$1.00. IB detects this unusually low price by comparing it to the reference price (around \$6.00) and classifies the observation as a multipack. It then generates a conversion factor equal to the ratio of the observed pack size to the true pack size (in this case $6/1 = 6$) and adjusts the volume and price so that the unit price is corrected.

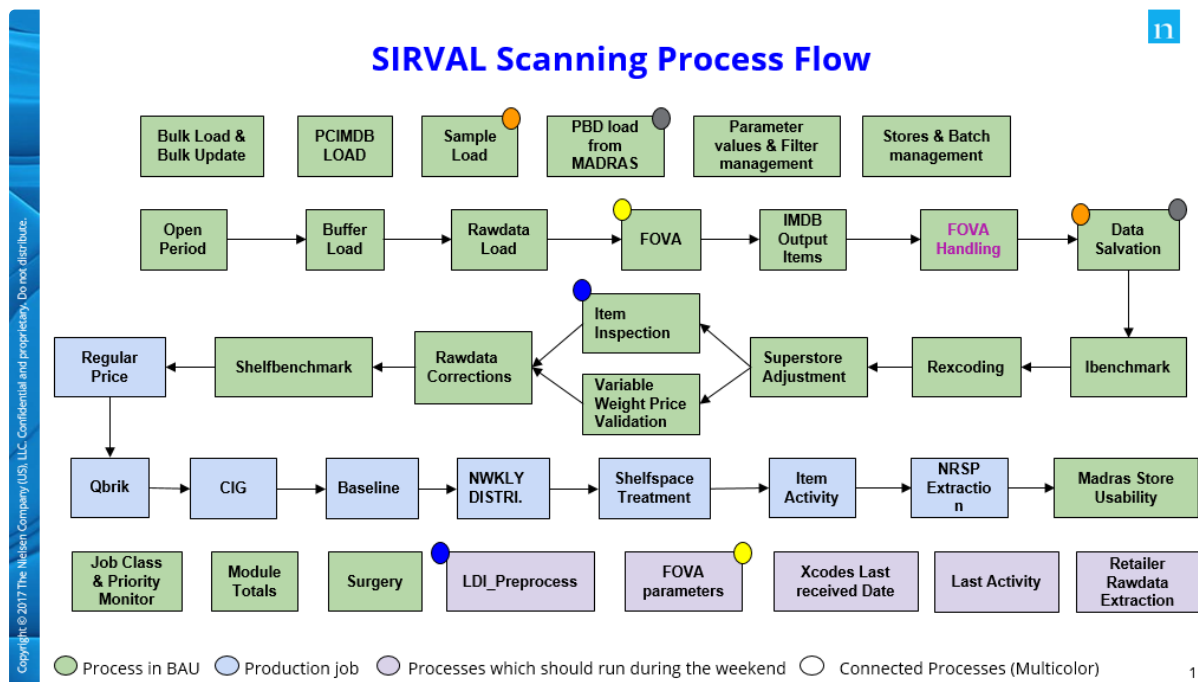
A high-level overview of IB's three pillars is:

1. **Calculation of a reference price** – IB updates an online estimate of the mean or median price for each item within a homogeneous group of stores called a **Price Group** (or **IBD**). This reference price and its variability are used to validate incoming observations.
2. **Multipack detection** – if an observation's price falls outside acceptable boundaries, IB uses product characteristics and statistical modelling to infer whether the item is a multipack and determine the most likely pack size.
3. **Cross-coding instructions (price suffixes)** – IB stores conversion factors in the PRICESUFFIXES table and applies them to future raw data so that volumes and prices are aligned.

Throughout this document, **raw price** refers to the value reported by the retailer. **Reference price** refers to the mean or median price stored in the IBENCHMARKS table, and **price suffix** refers to a code (e.g., P:I1-4) that instructs downstream processes how to adjust volumes and prices.

3 2. Where does the component belong? What part of the process is it located?

IBenchmark runs after **FOVA** (validation of raw data) and **data salvation**. It operates at the **shop/period** level: for each item and shop type, IB reads raw data, looks up or updates reference prices in the IBENCHMARKS table, performs validation and detection, then writes updates and price-suffix instructions. Corrected data feed into downstream processes such as **Qbrik**, **Baseline**, **CIG** and module totals. Proper upstream setup of **shop types** and **OGRDS characteristics** (weight/volume per unit, number in multipack, brand, packaging, promotional status) is essential for accurate detection.



4 3. Component/Method description

4.1 3.1 Manual suffix instructions and governance

Not all corrections are automatic. **Manual suffixes** can be applied or removed by operations teams when exceptional cases arise—such as when a retailer consistently misreports a pack size that the algorithm cannot detect, when a client requests a fixed conversion factor, or when a regional promotion alters pack sizes in ways that cannot be inferred from price distributions.

When to use manual suffixes – Before creating a manual suffix, operators should attempt to re-run IB and verify that OGRDS characteristics and parameters are correctly set. Manual suffixes are recommended only when the algorithm repeatedly misclassifies an item or when business rules override the statistical suggestion. They should not be used to “fix” occasional anomalies, as IB is designed to handle those automatically.

Creating and loading manual suffixes – The **Price Suffix Loader** provides four modes of operation: **Insert** (create new suffix), **Update** (modify conversion factor or bounds), **Delete** (remove a suffix) and **Replace** (delete existing and insert new). Operators prepare a text file with standard column delimiters containing at least the fields: period range (`AC_PERIODFROM` , `AC_PERIODTO`), store (`AC_NSHOPID`), item (`AC_CREF` or `F_NAN_KEY`), suffix (`AC_CREFSUFFIX`), conversion factor (`NC_CONV`), lower/upper bounds (`NC_PRICELB` , `NC_PRICEUB`), status (`AC_STATUS`) and optional description (`AC_CREFDESCRIPTIONSUFFIX`). After uploading, the loader offers a **Check Bad Rec** button to validate the records; errors such as missing fields, invalid conversion factors, `LB > UB` , `PERIODFROM > PERIODTO` or attempts to update non-existing records are flagged. Operators must correct these issues before final submission.

Governance guidelines – Manual suffixes have precedence over algorithmic suffixes, but IB still validates incoming prices against the manual bounds. If a raw price falls outside the manual limits, IB ignores the suffix and re-evaluates the observation using the original price. To ensure that manual suffixes are used judiciously and that underlying reference prices remain reliable, NIQ recommends the following governance principles:

- **Homogeneous and non-overlapping price groups** – Stores within the same price group (`IBD`) should exhibit similar pricing behavior. Overlapping groups or misconfigured shop types lead to unstable reference prices and more frequent manual overrides.
- **Sufficient panel size** – Each price group should contain at least **30 shops** to provide a robust sample for mean/median calculation.
- **Diverse composition** – Groups should include at least **three retailers** so that no single retailer dominates the reference price.
- **Quality input data** – Before resorting to manual corrections, verify that OGRDS characteristics (weight, pack size, brand, packaging) are properly coded and that `STORES` table shop types reflect real differences in price behaviour.

Operational considerations – Running IB and the price suffix loader concurrently on the same stores can cause conflicts. Operators should schedule manual uploads during periods when IB is not running and avoid launching Rexcode surgery at the same time. After manual suffixes are loaded, the next IB

run will apply them and update the `PRICESUFFIXES` and `XCODES` tables accordingly. The `STOREMONITOR` and `sys_activity_log` tables record the status of each run, including warnings when manual suffixes are ignored because prices fall outside bounds. Regular monitoring of these logs helps ensure that manual suffixes remain effective and that they are removed or updated when no longer needed.

- **Well configured in `STORES`** – shop types should reflect real price differences; mixing scanning and audit stores within a group should be avoided.

Homogeneity can be assessed by **ANOVA** or **coefficient of variation** tests ($CV = \sqrt{(\text{Var Sales}) / \text{Average Sales}}$). High CV values may indicate the need to merge or split groups. Automated scripts are under development to propose optimal price groups by analysing median distances and common items across groups.

4.2 3.2 Reference price calculation (mean/median update)

4.2.1 Inputs

- **Raw sales data** at item/shop/period level (values and units). Each record must include an item identifier (`NANKEY`), store identifier, period, and raw price (value / units).
- **Existing price suffixes** – conversion factors stored in the `PRICESUFFIXES` table. When IB reads a new observation it first checks whether a suffix applies; if so, it converts the raw price before validation.
- **Historical reference prices** from `IBENCHMARKS` – previous mean/median price, coefficient of variation (CV), quartiles (Q15, Q50, Q85), number of updates, and shock counters. These values provide the benchmark against which new observations are compared.
- **OGRDS/IMDB metadata** – product characteristics such as weight/volume per unit, number of pieces in a pack, brand, packaging type, and whether the product is promotional. These fields are loaded via `XTABCHARS` during `PC_IMDB` and used in multipack detection.
- **Configuration parameters** – thresholds and rules controlling learning rates, minimal prices, shock detection (parameters `DISTANCE_SHOCK_PRICE` and `PERIODS_OF_SHOCK_PRICE`), minimal pack size (`MIN_NPACK`), multipack detection mode (`MP_MODE`), median distance factors, and bin configuration. Default values exist but should be tuned to market conditions.

IB's reference price algorithm is **online**, meaning it updates continuously as each store's data arrive rather than waiting until the end of the period. The update procedure has two distinct phases depending on whether a product has been seen before:

- **First observation** – When IB encounters an item/store combination for the first time, it creates a row in the `IBENCHMARKS` table. The mean price (`NC_PRICE_MEAN`) is initialized to the **converted raw price**, provided that price exceeds the minimum threshold (`C_NC_PRICE_MIN` , default 0.01). A default coefficient of variation (`NC_PRICE_CV`) is assigned (0.08), and the three quartiles (Q15, Q50, Q85) are set equal to the converted price. These quartiles will later

define the dispersion of prices. If no price exists at the **NANKEY** level—for instance when a new item is found—IB optionally looks up broader keys such as UPC/EAN or aggregated **LAC/**
XCODE levels to seed the initial mean.

- **Ongoing updates** – Each subsequent observation prompts an incremental update. IB first determines a **learning rate**, a small number that controls how much weight to give the new observation relative to the existing mean. The learning rate is inversely related to the number of prior updates: early observations move the mean and quartiles significantly, whereas later ones have less influence. Using this rate, IB adjusts the quartiles toward the converted price: if the price is above Q50, Q85 is nudged upward and Q15 downward; if below Q50 the opposite occurs. After the quartiles are updated IB checks that they remain ordered ($Q15 < Q50 < Q85$); if not, they are reordered to maintain a valid distribution.

From the quartiles IB calculates a **variation** metric, conceptually similar to the coefficient of variation. The variation expresses how wide the price distribution is relative to the median; small values indicate a tight cluster of prices, while large values signal dispersed or volatile pricing. Based on this variation, IB computes **lower** and **upper bounds** for acceptable prices. For example, when variation is small (below 0.83) the bounds are narrow and centred on the median; when variation is moderate or high (between 0.83 and 1.2 or above 1.2) the bounds widen to accommodate greater dispersion. Prices falling outside these bounds do not immediately change the reference price; instead they trigger multipack or multi-piece detection. Observations within the bounds are used to update the mean or median: the mean is adjusted toward the new price by the learning rate, and the coefficient of variation is updated accordingly. To avoid drift, IB ensures that the updated mean remains within the Q15–Q85 interval; if it drifts outside, it is reset to the nearest quartile.

This iterative mechanism allows IB to track gradual changes in pricing while resisting outliers. The choice between **mean** and **median** reference price is configurable. Many countries use mean prices with quartiles because they offer intuitive lower and upper bounds. However, in categories with high price volatility or skewed distributions, median-based reference prices are more robust. Enhancements such as **bins** (grouping items by brand and weight to derive a group median) and **referential shoptype analysis** (comparing medians across shop types) provide alternative starting points when direct history is sparse.

To illustrate the update process, suppose an item has a current mean price of \$2.00, quartiles at \$1.90, \$2.00 and \$2.10, and has been updated 10 times. A new converted price of \$2.20 arrives. The learning rate might be 0.01 (1 divided by 100 updates), so IB increases Q85 slightly (toward \$2.20), decreases Q15 slightly (reflecting increased dispersion) and nudges Q50 upward. The variation may rise but remain below the high-variation threshold, so the acceptable upper bound might expand from \$2.20 to \$2.30. Because the new price lies within the bounds, IB updates the mean by 1 % of the difference: the new mean becomes $\$2.00 + 0.01 \times (\$2.20 - \$2.00) = \2.002 . Over time, dozens of observations will gradually pull the reference price toward the central tendency of the market.

4.3 3.3 Mean price stability (Dynamic vs. Stable Price)

Updating the reference price on every new store introduces a trade-off: while it keeps the benchmark responsive to market trends, it can also cause the mean to follow **short-term promotions** or **temporary shocks**. To strike a balance, IB maintains two separate benchmarks:

- **Dynamic Price (DP)** – this is the constantly updated mean or median price that reacts to each incoming observation. DP reflects the most recent market conditions and is used for day-to-day validation.
- **Stable Price (SP)** – this is a “memory” of the mean or median from previous weeks. It changes only when evidence indicates that the market has permanently shifted. SP shields the process from transient spikes caused by promotions, clearance sales or data anomalies.

IB compares DP and SP each week. If the relative difference $|DP - SP| / SP$ exceeds a threshold $P1$ (default 20 %), the algorithm considers the difference a **shock**. A shock may reflect a price promotion, a seasonal discount or an error in the data. Instead of immediately updating SP, IB starts a **shock counter**: it records how many consecutive weeks DP has diverged from SP by more than $P1$. If this divergence persists for more than $P2$ weeks (default 4), IB assumes that the change is no longer temporary and **updates SP** to the current DP; the shock counter resets. Otherwise SP remains unchanged, meaning that short promotions do not alter the stable benchmark.

When validating a new price, IB normally uses the **DP-based bounds** described earlier. However, if a shock is flagged, IB also computes **SP-based bounds**. The observation is validated against both sets of bounds; if it falls outside DP but within SP, IB may treat it as normal. This dual comparison is particularly important for **multipack detection**: if the Naïve Bayes algorithm is triggered, IB chooses whichever benchmark (DP or SP) yields a smaller absolute difference from the observed price. This prevents the algorithm from flagging an item as a multipack simply because DP dipped temporarily during a promotion.

Consider an example: Last month the dynamic price of a cereal pack was \$4.00 and the stable price was \$4.05. This week a supermarket runs a big promotion, and the dynamic price drops to \$3.50, a difference of about 12 %. Since $|3.50 - 4.05| / 4.05 \approx 14\%$ is less than the $P1$ threshold of 20 %, no shock is flagged. DP becomes \$3.50 and SP remains at \$4.05. Next week the promotion continues and DP remains at \$3.50. The shock counter increases but still does not exceed $P2$ (four weeks). During this period IB validates new observations against DP but also considers SP if a price lies near the boundary. If after four weeks the promotion ends and DP returns to \$4.00, the shock counter resets and SP remains unchanged. Only if the reduced price persists beyond four weeks will SP update to the new level, reflecting a permanent market shift.

4.4 3.4 Multipack detection

Once IB establishes the acceptable price range, it checks whether the converted raw price lies within the lower and upper bounds. Observations outside the range are suspicious and trigger **multipack detection**. Before describing the two-step logic, it is important to understand the **Naïve Bayes theorem** underpinning this methodology.

4.4.1 Naïve Bayes theorem and posterior probability

Bayes' theorem provides a principled way to update our belief about the class of an item—in this context, the pack size—after observing data. It tells us that the **probability of a specific pack size** given the observed characteristics and price is proportional to two pieces of information: (1) the **prior belief** about how common that pack size is and (2) the **likelihood** of observing the characteristics and price if the item truly belongs to that pack size. Under the **naïve independence** assumption, the

contributions of different features (for example, weight, brand, packaging and price) can be considered separately and then multiplied together. When comparing candidate pack sizes, the denominator of Bayes' theorem (which ensures probabilities sum to one) is the same for all and therefore can be ignored; the algorithm simply chooses the pack size with the largest product of prior and likelihood. This conceptual framework maps neatly to IB's two-step process: the prior comes from product characteristics (how often similar products are sold as single packs or multipacks), and the likelihood comes from how well the observed price matches the expected price for each pack size.

1. **Prior probability of being a multipack** – IB uses product characteristics to estimate how likely an item is to be sold in multipacks. Characteristics include weight/volume per unit, number in multipack (a numeric indicator of pack size), brand, packaging type and whether the product is under a promotional offer. Each characteristic is encoded in OGRDS via XTABCHARS. IB aggregates other items with identical characteristic values and enumerates possible pack sizes. The **prior probability** is simply the proportion of items with those characteristics that were historically sold in each pack size; this corresponds to Bayes' "prior belief" about the class.
2. **Likelihood based on observed price** – for each candidate pack size, IB models the relationship between price and pack size as a quadratic function whose coefficients are derived from the mean and standard deviation of historical prices. By plugging the **converted raw price** into these functions, IB evaluates how plausible the price is if the item were sold in that pack size; this is the **likelihood** component. Multiplying the likelihood by the prior for each class yields a relative score, and IB selects the pack size with the highest score. If the leading score falls below a threshold or the observation is ambiguous, IB returns a **DK** (don't know) or **OK** status, meaning no correction is applied.

Example – Imagine that, based on historical data, items with identical characteristics (brand, weight, packaging) are sold either as single packs or six-packs. Suppose 80 % of these items are single packs and 20 % are six-packs; thus our **prior belief** is that a new item with these characteristics is four times more likely to be a single pack than a six-pack. The average price of a single pack is about \$1 (with little variation), whereas the average price of a six-pack is about \$5.90. Now a retailer reports a raw price of roughly \$6. After adjusting for any existing suffix, IB asks: "How likely is it to see a price of \$6 if this is a single pack?" and "How likely is it if this is a six-pack?" Because \$6 is far above the single-pack average and very close to the six-pack average, the first likelihood is extremely small and the second is much larger. Multiplying these likelihoods by the respective priors gives relative scores: the single-pack score becomes tiny, and the six-pack score dominates. IB therefore infers that the item is a six-pack and generates a conversion factor of 6 to correct the data. This simplified example shows how the algorithm combines prior knowledge from product characteristics with evidence from the observed price to arrive at the most plausible pack size without needing to compute exact mathematical formulas.

If IB detects a multipack (status **MP**), it computes a **conversion factor** equal to the ratio of the predicted pack size to the coded size (e.g., predicted 6-pack vs. coded X1 yields factor 6). A corresponding price suffix (e.g., P:I1-6) is generated and stored. Key parameters controlling this step include:

- **MIN_NPACK** (default 3): Prevent corrections for negligible multipacks
- **MP_MODE** (0= no analysis; 1 = report only; 2 = analysis with suffix generation; 3 = analysis and report): Control whether multipack detection runs and whether suffixes are created
- **PRICE_UPDATE (Y/N)**: Controls whether reference prices are updated during multipack analysis

- **MIN_PRICE_TO_SKIP:** Raw prices below this threshold skip mean/median price update but still undergo multipack detection. This is to prevent unstable updates from near-zero prices
- **DISTANCE_SHOCK_PRICE** and **PERIODS_OF_SHOCK_PRICE** (0.2 and 4 by default): Determine when to flag shocks between DP and SP

4.5 3.5 Multi-piece analysis

Some modules or store types carry products whose “pack size” is not a single box or bottle but a **bundle of individual pieces**. Common examples include soup cubes sold in boxes of eight, bags of individually wrapped candies and multipacks of batteries. For such items, the product metadata in OGRDS includes a characteristic (often labelled *CONV FACTOR* or *Number of pieces*) indicating how many single units are in the pack. This characteristic is distinct from the **Number in Multipack – Base** used by multipack detection: it tells us how many pieces a retailer is supposed to sell together, not how many packs fit in a larger case.

Retailers sometimes ignore this instruction and sell pieces separately. When a consumer buys only one soup cube out of a pack of eight, the scanned **raw price** becomes one-eighth of the intended pack price. In such situations the standard multipack algorithm can mis-classify the observation because it looks for pack sizes like six or twelve derived from price distributions. To address this, IB includes a **multi-piece (MPE) analysis**. This routine uses the explicit number-of-pieces characteristic to compute a conversion factor that converts the transaction back to a single pack. MPE runs only if three prerequisites are satisfied:

1. **Module and store enabled** – the module (product category) must be listed in the IBENCHMARK_MODULES table and the store (or group of stores) must have the MPE_STORES parameter set to **Y**. This ensures that MPE is applied only where retailers are known to sell individual pieces.
2. **Conversion-factor characteristic defined** – a parameter CONV_FACTOR must point to an OGRDS characteristic containing the number of pieces in the pack. Without this characteristic MPE cannot compute a factor.
3. **Trigger condition** – MPE is invoked only when the multipack detection returns **DK** (don’t know) or **OK** but the price still falls outside the acceptable range. In other words, IB tries the standard multipack logic first; only if it cannot confidently classify the pack size does it fall back to MPE.

Once triggered, MPE follows a simple algorithm:

1. **Lookup the number of pieces** – IB reads the CONV_FACTOR characteristic for the item in the XTABCHARS table. This value—say, 60 for a carton of batteries or 21 for a throat-lozenge pack—tells how many pieces the retailer should sell together. IB computes a conversion factor as **1 divided by the number of pieces**. For 60 pieces the factor is 1/60, meaning that a transaction for a single piece must be multiplied by 60 to reconstruct the full pack.
2. **Apply the conversion factor** – IB multiplies the recorded units by the conversion factor and divides the sales value by the same factor to obtain a **converted price**. This operation is analogous to “putting the pieces back together”: one soup cube priced at \$0.25 becomes an eight-piece pack priced at \$2.00. Because MPE uses explicit piece counts, it is deterministic and does not involve statistical modelling.

3. **Validate the converted price** – IB compares the converted price to the lower and upper bounds derived from the reference price. If the converted price falls within the bounds, the observation is corrected and a **price suffix** is generated. If the converted price is still out of range, IB may accept the MPE correction if country-specific parameters allow a wider tolerance; otherwise it defers to **change maker**. Regardless of the outcome, IB logs both the initial DK decision and the MPE attempt so that analysts can audit the reasoning.

MPE generates suffixes of the form P:I1-<n>-MPE, where <n> is the number of pieces per pack. The prefix I1 indicates that the correction applies to the item level (as opposed to I10 for bundle packs). The suffix instructs downstream modules to multiply units by <n> and divide sales by <n>. Because MPE relies on explicit characteristics, its output is easy to interpret: the correction factor reflects the physical number of pieces in the pack. If the characteristic is missing (often coded as 1), if the store is not enabled for MPE or if the converted price remains suspicious, MPE passes the observation to **change maker** for a ratio-based calculation.

Example: In one case a retailer sells individual batteries from a 60-piece pack: the multipack algorithm cannot classify the observation, so IB reads the CONV_FACTOR characteristic (value = 60) and generates suffix P:I1-60-MPE. This restores the pack's value and units. Another example involves a candy pack coded as 21 pieces. The multipack algorithm returns DK, but MPE recognizes the 21-piece characteristic and issues suffix P:I1-21-MPE, correcting the price to match the benchmark. A more complex example shows that MPE may sometimes over-correct if the raw price is extremely low relative to the benchmark (e.g., a 24-piece battery pack scanned at a fraction of the price). In such cases IB logs the attempted correction and may defer to change maker.

4.6 3.6 Change maker

Change maker is a contingency method that kicks in when multi-piece analysis cannot provide a satisfactory correction. It is designed for two difficult scenarios:

1. **Missing piece information** – The product's CONV_FACTOR characteristic is empty or coded to 1, meaning that the number of pieces in the pack is unknown. This situation often arises when manufacturers or retailers omit the piece count or when country setups do not define the characteristic for certain modules.
2. **Persistently abnormal price** – A piece count is present and MPE has been applied, but the resulting converted price still lies outside the acceptable price range. In other words, MPE cannot reconcile the observation with the benchmark price because the raw price is too low or too high relative to the expected pack price.

In both cases the underlying question is: *"How many items would need to be grouped together for the observed price to make sense?"* Change maker answers this by comparing the benchmark price (dynamic or stable median) with the raw price and computing an implied number of units. The algorithm is conceptually straightforward:

1. **Eligibility check** – Change maker is executed only if multi-piece analysis is enabled for the store and module (MPE_STORES and IBENCHMARK_MODULES parameters) and if a global flag activates the ratio-based logic. Countries can restrict the feature to specific stores via a store-level parameter; the default behavior is to disable change maker unless explicitly enabled.

2. **Estimate pack size from price ratio** – IB divides the **benchmark price** by the **raw price**. The benchmark price is typically the dynamic median or mean for the item/price group and reflects what a single pack should cost. The raw price is what the retailer reported. The ratio therefore expresses how many times the observed price fits into the expected price. IB rounds this ratio to the nearest integer to infer the number of units inside the pack. For example, if the benchmark price is 2.94 and the raw price is 0.01, the ratio is $2.94 / 0.01 \approx 294$; IB infers that the retailer sold 1/294 of a standard pack. Similarly, a ratio of $42.0 / 1.5 \approx 28$ suggests that the observed item corresponds to a 28-piece pack.
3. **Derive conversion factor** – The conversion factor is simply the **reciprocal of the inferred number of units** (1/294 or 1/28 in the examples). Multiplying the raw units by this factor and dividing the value yields a converted price. IB then checks whether the converted price falls within the reference bounds. If it does, the correction is applied and a price suffix (e.g., P:I1-294-MPE) is generated. If the converted price still fails the bounds or if the inferred number of units is less than **2**, the correction is skipped to avoid unrealistic conversions.
4. **Safeguards and parameters** – Several parameters control the behaviour of change maker:
 - A store-level switch determines which stores can use change maker.
 - A global parameter allows countries to enable or disable the ratio-based path when the piece count characteristic is missing.
 - Minimum price thresholds (MIN_PRICE_TO_SKIP_VALIDATION, MIN_CORR_PRICE_TO_SKIP_APPLY) prevent the algorithm from applying corrections when raw prices are extremely small or when the resulting corrected price remains too low.
 - MPE_RULE_CONV may specify further rules such as maximum allowed pack size or rounding rules.

Examples – In one case the CONV_FACTOR characteristic is coded as 1 for Mentos gum, meaning the number of pieces is unknown. The benchmark price for a full pack is about 2.94, while the retailer reports 0.01. The ratio $2.94/0.01 \approx 294$ leads IB to infer a 294-piece pack and generate suffix P:I1-294-MPE. In another case, a candy pack coded as 21 pieces yields a converted price that is still outside the acceptable range; change maker computes the ratio $42.0/1.5 \approx 28$ and issues suffix P:I1-28-MPE, overriding the original 21-piece specification. These examples show that change maker can adapt to situations where retailers repackage products or sell subsets of a pack and where static characteristics are insufficient.

4.7 3.7 Bundle-pack analysis (BLP)

A **bundle pack** is a set of items that a retailer gathers or loosely binds together and sells as a unit, but which is not recognized as a formal multipack in the product metadata. Holiday gift boxes, “buy 9 get 1 free” promotions or spontaneous combinations of products fall into this category. Because these bundles are not coded in OGRDS there is no characteristic specifying the number of items. Without additional logic, such sales would inflate prices and volumes or be mistaken for multipacks.

Bundle-pack analysis runs only when the module and store are explicitly enabled. Two parameters control the feature:

- **BLP_ITEMS** – a list of module identifiers (product categories) where bundle-pack checks should be performed. The parameter value is typically a space-separated list of module codes;

the default is “N” (disabled). At least one module must be specified for IB to consider bundle packs.

- **BLP_STORES** – a list of store identifiers or a SQL query selecting stores to participate in bundle-pack analysis. Stores not in this list are excluded. The default value is also “N”, so bundle-pack checks are off unless activated.

Once these parameters are set, IB performs the following steps:

1. **Identify candidate observations** – For each item in an enabled module and store, IB retrieves the **mean (or median) benchmark price** from **IBENCHMARKS**. This price reflects what a single pack normally costs.
2. **Compute the bundle ratio** – IB divides the **raw price** (sales value divided by units) by the **mean price**. The resulting ratio indicates how many single packs are bundled together. For example, if the mean price of a candy bar is \$5 and a retailer sells a “10-for-\$50” bundle, the ratio is $50 \div 5 = 10$. IB rounds the ratio to the nearest integer to estimate the number of items in the bundle.
3. **Derive the conversion factor** – The conversion factor is the integer ratio itself. Because a bundle comprises multiple single packs, the units need to be multiplied by the ratio and the value divided by the ratio to express the transaction in terms of individual packs. IB generates a **price suffix** with pattern **P:I<ratio>-1-BLP** or similar (the **I** code indicates item-level correction). In our example the suffix is **P:I10-1-BLP**: the units are multiplied by 10 and the value is divided by 10, bringing the price per pack back to \$5.
4. **Validate the correction** – IB may impose additional safeguards similar to change maker: if the ratio is less than **2** (i.e., the raw price is lower than or equal to the mean price) or if the corrected price still falls outside the bounds, the algorithm skips the correction to avoid false positives. Parameters such as **MIN_PRICE_TO_SKIP_VALIDATION** and **MIN_CORR_PRICE_TO_SKIP_APPLY** also apply here; they prevent the creation of suffixes when raw prices are extremely low or when the converted price remains too low after correction. Bundle packs are intended for large multiples; a ratio of 10 or more is typical.

The output of bundle-pack analysis is a price suffix instructing downstream modules to treat the sale as multiple single packs. The suffix includes the ratio and a **BLP** flag to distinguish it from multipack or multi-piece corrections (e.g., **P:I10-1-BLP**, **P:I14-1-BLP**). Unlike multipack suffixes, which are based on combinations of OGRDS characteristics and statistical modelling, bundle-pack suffixes rely solely on the price ratio and require careful parameter configuration to prevent false positives.

Example – Suppose a store in module 558 sells gift bundles of 14 craft beers. The mean price for one beer is \$3.50. A retailer offers a holiday pack for \$49. The ratio is $49 / 3.50 = 14$. IB rounds to 14 and generates suffix **P:I14-1-BLP**. Raw units are multiplied by 14 and the value is divided by 14, aligning the corrected price with the benchmark. If the retailer sells a pack for \$48, the ratio is $48 / 3.50 \approx 13.7$, which rounds to 14; the same suffix is applied. This ensures that any bundle with roughly the right total price is treated consistently.

Bundle-pack analysis complements multipack and multi-piece checks by catching ad-hoc bundle promotions that are otherwise invisible in product metadata. However, because it depends solely on price ratios and not on product characteristics, it should be used judiciously and configured per module and per store to avoid misclassifying regular promotions or price fluctuations as bundles.

4.8 3.8 Price-suffix cross-coding instructions

A **price suffix** (sometimes called a **cross-coding instruction**) is an instruction that tells downstream processes how to convert raw sales data for an item when its reported value and units do not match. The price suffix is IB's way of "memorizing" a correction: once an issue is detected, the suffix persists so that future transactions from the same store and item are automatically converted without re-running the full detection logic. This persistence is important because retailers often continue to send mismatched data for several periods after an issue is discovered.

At a high level the **cross-coding workflow** follows four stages, each with clear inputs and outputs:

1. **Trigger** – When an observation's converted price falls outside the lower/upper bounds calculated from the reference price, IB flags the item for further analysis. If multipack, multi-piece, change maker or bundle-pack detection confirms that the item should be treated as a different pack size, the process prepares to generate a suffix. Inputs at this stage include the raw price, reference price and its variation, any OGRDS characteristics, and configuration flags controlling multipack modes.
2. **Generation** – IB constructs a suffix string such as `P:I1-6` (for a six-pack) or `P:I1-69-MPE` (for a sixty-nine-piece multi-piece correction) and computes the corresponding conversion factor (`NC_CONV = 6` or `69`). It determines appropriate price bounds around the converted price to ensure that the suffix is applied only when future raw prices remain consistent. The generation logic is implemented in a subroutine (`make_new_suffix`) that takes the detection result as input and returns a suffix record.
3. **Storage** – The generated suffix is written to the **PRICESUFFIXES** table and replicated into the **XCODE** table so that both IB and downstream processes (Qbrik, Rexcoding) can access it. At this point the suffix becomes part of the input dataset for subsequent periods. Output from this stage includes not only the suffix string and conversion factor but also price bounds, period range and status fields.
4. **Application and Monitoring** – During future IB runs, the algorithm first checks whether a suffix exists for the store/item. If it does and the raw price lies within the stored bounds, IB applies the conversion factor (multiplying units and dividing value) before validation. Converted observations update the reference price like normal. If a new price falls outside the bounds or the detection outcome is **OK** or **DK**, IB removes the suffix and reverts to normal detection. Throughout this stage, IB logs the decision in report tables and exposes warnings in `STORESTATS` or `sys_activity_log` so that operators can monitor suffix behaviour.

This step-by-step view underscores that cross-coding is not just a table of codes but a fully integrated loop: detection → generation → storage → application. The clarity of inputs (raw price, reference price, detection result) and outputs (suffix record, corrected data) allows the process to be reproduced or audited by newcomers.

Structure and storage – Each suffix is stored in the **PRICESUFFIXES** table with the following fields:

Field	Meaning
AC_PERIODFROM / AC_PERIODTO	The period range during which the suffix should apply. Typically the starting period is the one in which the issue was detected and the end period is left open until the suffix is removed.
AC_NSHOPID	Store identifier. Suffixes are defined at store/item level because different stores may have different packaging practices.
AC_CREF / F_NAN_KEY	Item identifier (CREF for scanning codes or NANKEY for internal product keys). Sometimes both are stored to link scanning and internal items.
AC_CREFSUFFIX / AC_CREFDESCRIPTIONSUFFIX	The suffix string. It has the general pattern <code>P:I<packs>-<divisor></code> plus an optional flag (e.g., <code>-MPE</code> or <code>-BLP</code>). <code>P</code> stands for price, <code>I</code> denotes item level, the first number indicates how many single packs are contained in the raw data (e.g., 6, 60, 294) and the second number indicates the number of reported packs (normally 1). Flags specify which algorithm generated the suffix: <code>MPE</code> for multi-piece, <code>BLP</code> for bundle pack, etc.
NC_CONV	Conversion factor applied to volumes and values. If the suffix is <code>P:I1-6</code> , <code>NC_CONV</code> is 6; if the suffix is <code>P:I1-69-MPE</code> , <code>NC_CONV</code> is 69; if the suffix is <code>P:I10-1-BLP</code> the factor is 10. The factor is always the ratio of the recommended pack size to the reported pack size.
NC_PRICELB / NC_PRICEUB	Lower and upper price bounds used to determine whether the suffix is still valid. These come from the reference price and variation calculated during detection.
AC_STATUS / FIELDS in PRICESUFFIXES	Flags indicating whether the suffix is active, manually applied, removed, or superseded.

Suffixes are also replicated into the `XCODES` (or `XCODE`) table used by Qbrik. The `XCODES` table stores similar information but at a code level and is used by raw data correction processes to apply conversion factors on the fly.

The table design reflects the twofold purpose of cross-coding: to **remember the pack size correction** and to **control its application**. Fields such as `NC_PRICELB` and `NC_PRICEUB` implement a simple conditional rule: only apply the suffix when the raw price falls between these bounds. Likewise, the period fields define a temporal scope: operators can set `AC_PERIODTO` to stop applying a suffix after a certain week.

Generation – After IB determines that an observation is a multipack or similar, a piece of code (`make_new_suffix`) constructs the suffix string based on the suggested pack size and algorithm status. The process then calls `Create_PSI` to insert a new record into `PRICESUFFIXES` and `XCODES` . Only certain multipack detection modes (`MP_MODE` = 2 or 3) lead to suffix generation. IB sets the price bounds around the converted price so that future prices are checked before applying the conversion.

Behind the scenes, the code checks configuration flags such as `v_nc_ps_generate_module` and `v_nc_multipack` (from `PARAM_PROCESS`) to decide whether to create a suffix. This ensures that cross-coding is applied only for items and modules explicitly enabled. The new suffix is usually defined at **shop/item** level (so that different stores can have different corrections), but alternative specificities (e.g., chain or chain-module) may be supported in some implementations.

Application – When a new observation arrives, IB first checks whether a suffix exists for the store/item. If it does, and the **raw price** falls within the stored lower and upper bounds, IB applies the conversion factor: it multiplies units by `NC_CONV` and divides the sales value by the same factor. The converted price is then validated against the reference price. This ensures that even if retailers continue to report multipack data, the downstream modules see corrected volumes and prices without repeated detection.

Removal or update – Suffixes should not live forever. IB monitors each store/item combination and re-evaluates whether the suffix is still needed. If a new price, after applying the existing suffix, lies outside the stored bounds, IB removes the suffix. More importantly, if the multipack detection reruns on a store with a suffix and the outcome is **OK** (price matches expected pack size) or **DK** (indeterminate), the suffix should be dropped. Continuing to apply a factor when the price is correct would distort volumes. The process therefore recommends dropping suffixes in both OK and DK situations, not just when the price falls outside the boundaries.

Manual management and validation – Operators can manually insert, update or delete suffixes using the **Price Suffix Loader** interface (Op Setup → Loaders → Process Upload → Price Suffix Loader). This is necessary when the algorithm cannot correctly detect a multipack or when business rules require an override. The loader accepts a text file containing the necessary fields and provides modes for insert, update, delete or replace. Recent enhancements validate manual inputs: the system rejects records where `NC_CONV` < 0.0001, NULL, blank or zero; it checks that the lower bound is less than the upper bound; it ensures that records exist before updates; and it warns when descriptions are missing for LAC codes. Bad records are listed so that users can correct them before submission. Manual suffixes are flagged in the table and have precedence unless the algorithm subsequently deems them unnecessary.

Safeguards and parameter controls – To prevent the creation of unreasonable suffixes, several parameters provide guard rails. For example, `MIN_PRICE_TO_SKIP_VALIDATION` specifies a minimum raw price (default 0.01) below which IB skips all validation and suffix generation; this avoids generating suffixes for negligible prices such as \$0.01 which could produce large conversion factors. Similarly, `MIN_CORR_PRICE_TO_SKIP_APPLY` defines a minimum corrected price (default 0.01) below which a suffix is not applied, and `MPE_RULE_CONV` determines whether multi-piece corrections proceed when the conversion factor would be extremely large or zero. These parameters can be set at store specificity and are maintained by operators via the parameters table. If a suffix

would result in a conversion factor that rounds to zero (after four decimal places), the correction is skipped altogether.

IB also supports an **item-override mechanism** via the `EXCLUDE_IB_CORRECTIONS` table. When a client requests that a particular item be excluded from IB corrections (e.g., because the multipack correction is managed upstream), operations can insert the item into this table. IB then skips detection and suffix generation for that item in all stores. This feature was introduced to handle cases where corrections need to be fixed at manual price boundaries. Items can be added or removed from the table via bulk upload, and no parameter activation is required once the table is populated.

Additionally, restrictions exist for **LAC codes** (large aggregated codes): when uploading manual suffixes, the description suffix (`AC_CREFDESCRIPTIONSUFFIX`) must begin with `D:` to ensure that the suffix applies correctly to LAC-type items. The loader rejects LAC suffixes with blank or invalid description fields and displays an informative message. Different validation paths are used when the `AC_XCODEGR` field indicates shop code versus non-shop code level.

Interaction with other processes – During raw data correction (Qbrik and surgery), the system reads suffixes from `XCODER` and applies the conversion factor to each transaction. At the same time it ensures that no two processes (e.g., IBenchmark and Rexcoding) run concurrently on the same data to avoid conflicting instructions. Monitor tags in the `STOREMONITOR` table prevent duplicate runs, and `STORESTATS` and report tables provide transparency on how many items were corrected by suffixes versus fresh detections. When IB publishes its results to downstream platforms (eDIVE, Baseline, CIG), these suffixes ensure consistency across systems.

Example – Consider a battery item where multipack detection determines that the correct pack size is 6 instead of 1. IB generates suffix `P:I1-6` with conversion factor 6 and stores it in

`PRICESUFFIXES` with bounds [1.30, 1.70]. Next period, the retailer reports a raw price of \$1.50. Because 1.50 lies within the bounds, IB multiplies the units by 6 and divides the value by 6, converting the price back to \$9.00. The suffix remains active. In a later period, the retailer reports a raw price of \$9.10, which after applying the suffix yields \$54.60, far outside the bounds. IB removes the suffix and re-evaluates the observation; if multipack detection returns **OK**, no new suffix is created. This mechanism ensures that corrections persist only as long as they are needed.

4.9 3.9 Median price enhancements and bins approach

The standard mean/median update can produce unstable reference prices for low-distributed items or when new products arrive. IB therefore includes a suite of **median-price enhancements** to make the benchmarks more robust and to harmonize prices across shop types. These enhancements are implemented outside the core IB run as a **Python utility** that generates correction files for the `IBENCHMARKS` table. Three complementary components address different issues:

1. **Referential shoptype analysis** – This part of the script analyses differences in median prices across **IBDs** (price groups). For each module or category, the algorithm selects a **reference IBD** –usually the group with the most items and updates—and compares the median price of each item in other IBDs against the reference. Several parameters guide this analysis:

- `excl_ref` – marks IBDs that should not serve as a reference (e.g., audit groups or test shop types). When set to **Y**, the IBD is excluded from both the reference list and comparisons.
- `excl_ibd` – excludes certain IBDs from the analysis entirely. This is useful if some shop types are obsolete or under development.
- `sel_mod` – restricts the analysis to selected modules or categories. If set to **Y**, only modules listed in a separate configuration are processed.
- `old_per` – defines how old a median price can be to count as a reference; items with `NC_PERLAST` older than `current period - old_per` are ignored (default 12 weeks).
- `min_upd` – minimum number of updates (`NC_UPDATES`) required for an item to be considered a reliable reference (default 20).

Once the parameters filter the data, IBDs are **ranked** by the number of items or updates. For each item present in multiple IBDs, the script calculates the **median ratio** between the non-reference and reference IBDs; if the ratio differs too much from 1 (meaning the medians diverge), the item is flagged. Quartile comparisons provide additional checks: an item is marked as problematic if its Q15–Q85 range, when adjusted by the median ratio (`mod_f`), does not overlap the reference range or if the ratio of Q50 values exceeds a threshold. The output of this component consists of two Excel sheets:

- `Ib_corr` – items where the script proposes an updated median price. These items are unique to a single shoptype or have consistent corrections across shop types.
 - `corr_to_be_checked` – items requiring manual review because multiple shop types suggest different corrections or because the proposed median price is far from the original. Two additional parameters, `corr_lim_param` (ratio limit) and `currency_param` (absolute currency difference), determine when a proposed median price is considered too distant from the current one.
2. **Bins concept for low-distributed items** – Low-distributed items may never be flagged as outliers because they appear in only a few periods. The **bins** method addresses this by grouping items with similar characteristics to estimate a reasonable median price. The steps are:
- **Parameter selection** – the user chooses a `ratio_param` (default 3) that sets how wide the acceptable range should be around the normalized price. A smaller value yields tighter bounds and more outliers.
 - **Data preparation** – for each module/brand combination, the script reads **weight**, **brand**, **module** and **number of pieces** from OGRDS. Items lacking these fields are removed. A **normalized price** is computed as the raw price divided by weight (or number of pieces); this expresses the cost per unit weight or unit piece.
 - **Bin formation** – items are grouped into bins by **module and brand**. Each bin must contain at least two items. The script calculates the **mean normalized price** (`NORM_MEAN`) within each bin and then reverts back to an estimated price (`NC_CLUST_MEAN`) by multiplying by

the typical weight or piece count. Confidence intervals are derived using the inter-quartile range (IQR) for the normalized prices.

- **Outlier detection** – any item whose normalized price lies outside the bin's confidence interval is flagged. Additionally, the script compares the ratio of the **IB median price** to `NC_CLUST_MEAN` ; if this ratio is greater than `ratio_param` (e.g., >3) or less than its reciprocal (e.g., <1/3), the item is marked as exceptional. To avoid excessive corrections, items are not considered exceptional if this ratio lies between 0.5 and 2..

The bins component produces an Excel file (`qnr_output_bins_DD-MM-YYYY.xlsx`) listing low-distributed items and their suggested median prices. This output is combined with the referential shoptype results to produce the final correction file.

3. **Total Country IBD** – When a new item appears in a shop type that has no existing record in `IBENCHMARKS` , the **Total Country IBD** provides a fallback reference. Each week, after IB completes, the script scans raw data (with all price suffixes applied) from the last **four weeks** and computes a **weighted average** of mean prices, medians and quartiles. Items appearing in all four weeks are weighted 50% for the current week, 25% for the previous week, 10% for the third week and 5% for the fourth week; items appearing in fewer weeks use weights such as 70/20/10 or 80/20, depending on their presence. These weighted statistics populate a `TotalCountry_IBD` table that IB consults whenever an item is missing from its regular shop type. When IB processes a new observation and cannot find it in the shop-type IBD, it looks up the total country IBD and applies the same validation and multipack detection logic. This mechanism mitigates the risk of uncorrected mismatches during retailer onboarding or when retailers are excluded from median price calculation.

Together, these three components enhance the robustness of median price estimates, reduce volatility for low-distributed items and harmonise price behaviour across shop types. They operate offline as a supplemental script and output correction files that feed into the `IBENCHMARKS` update cycle.

5 4. Inputs

Input	Description	Notes
Raw sales data (value, units)	Transactional data at item, store and period level. The raw price is computed as $\text{value} \div \text{units}$.	Must be loaded via Bulk/Buffer Load and FOVA; includes fields like <code>AC_ITEMKEY</code> , <code>AC_STOREKEY</code> , <code>AC_PERIODID</code> , <code>NC_VALUE</code> , <code>NC_UNITS</code> .
Existing price suffixes (PRICESU FFIXES)	Table of conversion factors previously generated by IB. Each entry contains period, store, item, suffix code and <code>NC_CONV</code> .	When a new observation arrives, IB checks this table and applies the factor before validation.
IBENCHM ARKS table	Stores historical reference prices: mean or median, CV, quartiles (Q15, Q50, Q85), number of updates (<code>NC_UPDATES</code>), first and last period (<code>NC_PERFIRST</code> , <code>NC_PERLAST</code>), stable price (<code>NC_STABLE_PRICE</code>), shock counter (<code>NC_SHOCK_COUNTER</code>).	The table is updated every run; stable price and history support shock handling and back-data processing. A separate <code>IBENCHMARKS_HISTORY</code> table may capture historical mean prices period-wise.
OGRDS/ IMDB metadata (XTABCH ARS)	Holds characteristic values for each item: weight/ volume per unit, number in multipack, brand, packaging type, promotional flag, etc.	Essential for multipack detection and multi-piece analysis. Populated during <code>PC_IMDB</code> load.
STORES table	Defines shop types (<code>SHOPTYPE</code>) and price groups (<code>IBD</code>). Stores in the same group should share similar pricing behaviour.	IB uses <code>SHOPTYPE</code> to group stores; misconfiguration can cause incorrect reference prices and suffixes.
Configura tion parameters	Settings controlling the algorithm: learning rate (<code>C_NC_PRICE_ALPHA</code>), minimal price (<code>C_NC_PRICE_MIN</code>), minimal CV, minimal pack size (<code>MIN_NPACK</code>), multipack detection mode (<code>MP_MODE</code>), shock thresholds (<code>DISTANCE_SHOCK_PRICE</code> , <code>PERIODS_OF_SHOCK_PRICE</code>), bin and referential shoptype parameters, median distance factor, total country IBD switch, etc.	Parameters are stored in <code>PARAMETERS</code> or <code>PARAM_PROCESS</code> tables; they may be set at global, store or module specificity.

6 5. Outputs

IB generates several outputs consumed by downstream processes

Output	Description
Updated IBENCHMARK KS table	Contains the latest mean/median price, CV, quartiles, number of updates, stable price and flags for each item/price group. If mean price updates are disabled (PRICE_UPDATE=N) or history mode is active, updates may be written into IBENCHMARKS_HISTORY .
Lower and upper price bounds	Derived from quartiles and variation; determine whether an observation is “normal” or suspicious.
Multipack detection results	For each suspicious observation: suggested pack size, status (MP, OK, DK), multipiece flag (MPE), change maker flag, bundle-pack flag and any computed ratios.
Price-suffix instructions	Stored in PRICESUFFIXES and XCODE tables. Each instruction includes the suffix string and conversion factor. Suffixes are applied during Qbrik to convert raw data; they are removed when prices return to normal.
Raw data updates	Volumes are multiplied by NC_CONV and values are divided by NC_CONV so that downstream modules see corrected volumes and prices.
Correction files	Intermediate reports from referential shoctype analysis and bins approach; identify items or price groups requiring manual review or median corrections.
History tables	IBENCHMARKS_HISTORY captures historical mean/median prices for back-data reprocessing or retailer onboarding; Total Country IBD tables hold fallback reference prices for new items.
Reports	Summaries of each run indicating number of MP, OK, DK cases, suggested pack sizes, raw vs. converted prices, shock flags, and any manual suffix overrides. The sys_activity_log lists warnings (e.g., low ACV, duplicate runs).

7 6. Best practices and checklist

To ensure reliable results, the following checklist is recommended:

1. **Verify Shoptype Definitions** – confirm that stores in the same `SHOPTYPE` exhibit similar price behaviour and that groups contain at least 30 stores. Avoid mixing scanning and audit stores; ensure multiple retailers per group.
2. **Set Up OGRDS Characteristics** – weight/volume per unit, number in multipack, brand, packaging and promotion fields must be correctly coded in `XTABCHARS` during `PC_IMDB` load. Missing or incorrect characteristics lead to wrong multipack detection.
3. **Tune Parameters** – adjust `MP_MODE` (analysis/report behaviour), `MIN_NPACK`, learning rate (`C_NC_PRICE_ALPHA`), minimal price (`C_NC_PRICE_MIN`), shock thresholds, bin parameters, multi-piece and change maker switches, and bundle-pack lists based on country needs. Consult the `PARAMETERS` tables and default values.
4. **Prepare Raw Data** – run FOVA and data salvation; ensure the period is open; avoid rerunning IB for the same shop/period without nullifying prior results; use the splitted-store checkbox if stores change shoptype mid-period.
5. **Submit IB in SOL** – navigate to `Process Submission` → `Item Level Validation` → `Item Benchmark`, select stores and modules, and submit. Monitor run status and logs. Price suffixes may later be uploaded manually via `Op Setup` → `Loaders` → `Process Upload` → `Price Suffix Loader` if corrections are needed.
6. **Monitor Outputs** – review `sys_activity_log` for warnings (low ACV, duplicate runs); check reports showing MP, OK, DK, MPE, BLP outcomes and suggested pack sizes; examine correction files from bins and referential analyses for items requiring manual intervention.
7. **Validate Downstream Effects** – ensure price suffix instructions are applied during Qbrik and confirm that corrected prices and volumes flow into Baseline, CIG and other modules. For new items, verify that country-level or history tables provide appropriate reference prices.